

TP2 – Code Pratique : Détection d’objets 3D

Table des matières

1	Introduction	2
2	Exercice 1 : Configuration des communications ROS 2	2
2.1	Objectif	2
2.2	Ce que vous devez remplir	2
2.3	Indices	2
3	Exercice 2 : Extraction de la matrice intrinsèque	2
3.1	Objectif	2
3.2	Rappel théorique	3
3.3	Ce que vous devez remplir	3
3.4	Vérification	3
4	Exercice 3 : Conversion de l’unité de profondeur	3
4.1	Objectif	3
4.2	Rappel théorique	3
4.3	Ce que vous devez remplir	4
5	Exercice 4 : Modèle sténopé (Projection inverse)	4
5.1	Objectif	4
5.2	Rappel théorique	4
5.3	Ce que vous devez remplir	4
6	Exercice 5 : Calcul des dimensions physiques	4
6.1	Objectif	4
6.2	Rappel théorique	4
6.3	Ce que vous devez remplir	4
7	Exercice 6 : Publication du message ROS 2	5
7.1	Objectif	5
7.2	Rappel théorique	5
7.3	Ce que vous devez remplir	5
8	Conclusion	5
9	Résumé des solutions	5

1 Introduction

Dans ce TP, vous allez compléter progressivement un squelette de code de détection 3D en ROS2. Le fichier `detector_3d.py` vous est fourni. Chaque exercice correspond à une partie de la théorie vue précédemment et vous indique précisément ce que vous devez remplir. Dans ce Tp vous allez comprendre et mettre en place de la détection 3D avec une caméra de profondeur.

2 Exercice 1 : Configuration des communications ROS 2

2.1 Objectif

Configurer les subscribers et publishers pour récupérer les données de la caméra RealSense et publier les résultats de détection.

2.2 Ce que vous devez remplir

Dans la méthode `__init__`, trouvez les zones # À COMPLÉTER et remplissez :

1. **Le subscriber** `sub_info` : le type de message et le nom du topic de calibration.
2. **Le subscriber** `sub_rgb` : le type de message et le nom du topic image couleur.
3. **Le subscriber** `sub_depth` : le type de message et le nom du topic image de profondeur.
4. **Le publisher** `pub_image` : le type de message pour publier une image annotée.
5. **Le publisher** `pub_points` : le type de message pour publier des coordonnées 3D.

2.3 Indices

Pour trouver les noms des topics disponibles :

```
ros2 topic list | grep camera
```

Pour voir le type d'un message :

```
ros2 interface show sensor_msgs/msg/CameraInfo
```

Les types de messages dont vous aurez besoin sont déjà importés en haut du fichier.

3 Exercice 2 : Extraction de la matrice intrinsèque

3.1 Objectif

Dans la méthode `callback_camera_info`, extraire les paramètres f_x , f_y , c_x et c_y du message `CameraInfo` reçu.

3.2 Rappel théorique

La matrice intrinsèque K est structurée comme suit :

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (1)$$

Dans le message ROS `CameraInfo`, ces valeurs sont stockées dans un tableau linéaire `k` de 9 éléments :

Elle se présente sous une forme 3×3 :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y : Les distances focales exprimées en pixels. Elles décrivent le grossissement de la lentille sur les axes X et Y.
- c_x, c_y : Les coordonnées en pixels du **centre optique** (l'endroit où l'axe de la lentille traverse le capteur, proche du centre de l'image).

Dans ROS 2, pour simplifier le transfert réseau, cette matrice est aplatie dans un tableau à une seule dimension de 9 éléments : `[fx, 0, cx, 0, fy, cy, 0, 0, 1]`.

FIGURE 1 – Structure du tableau `k` dans le message `CameraInfo`

3.3 Ce que vous devez remplir

Dans `callback_camera_info`, remplissez les quatre variables `self.fx`, `self.fy`, `self.cx`, `self.cy` en lisant les bons indices du tableau `msg.k`.

3.4 Vérification

```
ros2 topic echo /camera/camera/color/camera_info | grep -A 10 "k:"
```

4 Exercice 3 : Conversion de l'unité de profondeur

4.1 Objectif

Dans la méthode `pixel_to_3d`, calculer la profondeur Z en mètres à partir du patch de pixels de profondeur.

4.2 Rappel théorique

La RealSense D435 fournit la profondeur en **millimètres** au format `uint16`. Pour convertir en mètres, il faut diviser par 1000. La valeur de la profondeur est fournie en `uint16` car `uint8` ne permettrait d'aller que jusqu'à 25.5cm de distance (valeur de 0 à 255 mm) alors que `uint16` permet d'aller jusqu'à 65 mètres (valeurs de 0 à 65 535 mm).

Pour plus de robustesse face au bruit du capteur, on utilise la **médiane** plutôt que la moyenne sur la fenêtre de 10×10 pixels déjà extraite dans le code.

4.3 Ce que vous devez remplir

Calculez Z en une ligne : prenez la médiane du tableau `valid` et convertissez en mètres.

5 Exercice 4 : Modèle sténopé (Projection inverse)

5.1 Objectif

Toujours dans `pixel_to_3d`, calculer les coordonnées spatiales X et Y en mètres à partir du pixel (u, v) et de la profondeur Z .

5.2 Rappel théorique

Le modèle sténopé relie l'espace 3D à l'image 2D. La projection inverse donne :

$$X = \frac{(u - c_x) \cdot Z}{f_x} \quad Y = \frac{(v - c_y) \cdot Z}{f_y} \quad (2)$$

5.3 Ce que vous devez remplir

Calculez X et Y en appliquant les formules ci-dessus avec `self.fx`, `self.fy`, `self.cx`, `self.cy`.

6 Exercice 5 : Calcul des dimensions physiques

6.1 Objectif

Dans la méthode `taille_reelle`, calculer la largeur et la hauteur réelles de l'objet détecté en mètres.

6.2 Rappel théorique

On applique le théorème de Thalès à l'optique. La taille réelle d'un objet se calcule depuis sa boîte englobante en pixels :

$$W = \frac{(x_2 - x_1) \cdot Z}{f_x} \quad H = \frac{(y_2 - y_1) \cdot Z}{f_y} \quad (3)$$

6.3 Ce que vous devez remplir

Calculez `largeur_m` et `hauteur_m` en appliquant les formules ci-dessus. Les coordonnées (x_1, y_1, x_2, y_2) de la boîte YOLO sont passées en paramètre de la méthode.

7 Exercice 6 : Publication du message ROS 2

7.1 Objectif

Dans `callback_rgbd`, remplir et publier le message `PointStamped` avec les coordonnées 3D calculées.

7.2 Rappel théorique

Un message `PointStamped` contient deux champs :

- `header` : le timestamp et le frame de référence (toujours à copier depuis le message reçu)
- `point` : les coordonnées `x`, `y`, `z` en mètres

7.3 Ce que vous devez remplir

Dans la boucle de détection, remplissez l'objet `pt` de type `PointStamped` avec les valeurs `X`, `Y`, `Z` calculées, puis publiez-le.

8 Conclusion

Vous avez maintenant un code capable de détecter des objets en 3D. Vous avez, leur classe (si la classe est dans le modèle), leur distance et leur taille.

9 Résumé des solutions

Exercice 1

- `sub_info` : `CameraInfo`, `'/camera/camera/color/camera_info'`
- `sub_rgb` : `Image`, `'/camera/camera/color/image_raw'`
- `sub_depth` : `Image`, `'/camera/camera/depth/image_rect_raw'`
- `pub_image` : `Image`
- `pub_points` : `PointStamped`

Exercice 2

```
self.fx = msg.k[0]
self.fy = msg.k[4]
self.cx = msg.k[2]
self.cy = msg.k[5]
```

Exercice 3

```
Z = np.median(valid) / 1000.0
```

Exercice 4

```
X = (u - self.cx) * Z / self.fx  
Y = (v - self.cy) * Z / self.fy
```

Exercice 5

```
largeur_m = (x2 - x1) * Z / self.fx  
hauteur_m = (y2 - y1) * Z / self.fy
```

Exercice 6

```
pt = PointStamped()  
pt.header = rgb_msg.header  
pt.point.x = X  
pt.point.y = Y  
pt.point.z = Z  
self.pub_points.publish(pt)
```